Stored Procedures

- Introducción y conceptos
- Estructura
- Parámetros
- Control de Flujos
- Ejecución y Resultados
- Ejercicios

Stored Procedure

Un **Stored Procedure (SP)** o *Procedimiento Almacenado* es un **conjunto de instrucciones SQL** que se compilan y se guardan directamente dentro del servidor de la base de datos bajo un nombre específico

Se pueden invocar ejecutando una sola línea de comando (por ejemplo: EXEC registrar_venta(...)), pudiendo recibir **parámetros de entrada** y devolver **resultados o parámetros de salida**.

Los Stored Procedures sirven para:

- **Encapsular la lógica de negocio directamente en la base de datos:** Evita tener que escribir la misma consulta SQL compleja en múltiples aplicaciones o partes del código.
- **Ejecutar operaciones compuestas o transaccionales:** Permitir que varias operaciones (por ejemplo: INSERT, UPDATE, DELETE) se ejecuten juntas como una unidad de trabajo atómica (garantizando consistencia/ACID).
- **Automatizar procesos repetitivos:** Tareas periódicas de mantenimiento, procesamiento de lotes (batching) o generación de reportes.

Estructura de SP

Estructura y Variables:

- Delimiter (// y ;)
- Begin
- Declare
- Set

```
1   DELIMITER //
2   CREATE PROCEDURE sp_contar_alquileres_cliente(
3       IN  p_customer_id INT,
4       OUT p_total_alquileres INT
5   )
6   BEGIN
7           DECLARE v_cantidad_temporal INT DEFAULT 0;
8
9       SELECT COUNT(*)
10      INTO v_cantidad_temporal
11      FROM rental
12      WHERE customer_id = p_customer_id;
13
14      SET p_total_alquileres = v_cantidad_temporal;
15  END //
16  DELIMITER ;
17
18
```

Parámetros (IN, OUT, INOUT)

- **IN (Entrada):** Una variable que le "envías" al procedimiento (sólo lectura dentro del SP).
- **OUT (Salida):** Un parámetro que el procedimiento va a llenar y devolverte al finalizar.
- **INOUT (Entrada/Salida):** Un valor que entra, el procedimiento lo modifica y te devuelve el valor transformado.

```
1   DELIMITER //
2   CREATE PROCEDURE sp_contar_alquileres_cliente(
3       IN  p_customer_id INT,
4       OUT p_total_alquileres INT
5   )
6   BEGIN
7           DECLARE v_cantidad_temporal INT DEFAULT 0;
8
9       SELECT COUNT(*)
10      INTO v_cantidad_temporal
11      FROM rental
12      WHERE customer_id = p_customer_id;
13
14      SET p_total_alquileres = v_cantidad_temporal;
15  END //
16  DELIMITER ;
17
18
```

Control de Flujo (IF, CASE, WHILE)

IF
Segmentar y clasificar datos según condiciones

```
IF p_total_alquileres > 30
THEN SET p_categoria = 'Cliente VIP';
 ELSE
            IF p_total_alquileres BETWEEN 10 AND 30
            THEN SET p_categoria = 'Cliente Regular';
            ELSE SET p_categoria = 'Cliente Nuevo';
END IF;
```

CASE
Ideal para evaluar opciones múltiples fijas:

```
CASE p_rating
   WHEN 'G' THEN SET v_descuento = 0.20;
   WHEN 'PG' THEN SET v_descuento = 0.10;
   ELSE SET v_descuento = 0.00;
END CASE;
```

WHILE
Muestra cómo iterar mientras se cumpla una condición:
SQL

```
DECLARE v_contador INT DEFAULT 1;
WHILE v_contador <= 5
DO
SET v_contador = v_contador + 1;
END WHILE;
```

Ejecución y Resultado SP

Un procedimiento con parámetros OUT **no muestra directamente una tabla**, sino que guarda el dato en una **variable de sesión de usuario** (identificada con @).

```
18 •    CALL sp_contar_alquileres_cliente (130,@resultado);
19 •    SELECT @resultado AS TotalAlquileres;
20
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TotalAlquileres
24

EJERCICIO CREACIÓN SP

- MESA 1: **Evaluador de Fidelidad de Clientes**
- MESA 2: **Verificador de Stock y Estado de Cliente**
- MESA 3: **Calculador de Tarifa por Clasificación (Rating)**
- MESA 4: **Generador de Cupones de Descuento**

EJERCICIO MESA 1

Evaluador de Fidelidad de Clientes

Crear el procedimiento *sp_evaluar_fidelidad_cliente*. Debe recibir el ID de un cliente y devolver su categoría junto con un porcentaje de descuento asignado (debe ser en decimal):

Lógica a implementar:

- Contar la cantidad total de alquileres del cliente en la tabla rental.
- Si alquiló más de 30 películas: Categoría 'VIP' y 0.20de descuento.
- Si alquiló entre 15 y 30 películas: Categoría 'FRECUENTE' y 0.10de descuento.
- Si alquiló menos de 15 películas: Categoría 'REGULAR' y 0.00de descuento.

```
CALL sp_evaluar_fidelidad_cliente(1, @categoria, @descuento);
SELECT @categoria, @descuento;
```

EJERCICIO MESA 2

Verificador de Stock y Estado de Cliente

Crear el procedimiento sp_validar_alquiler_express. Debe recibir el ID del cliente ,el ID de la película y la tienda y devolver un código de estado junto con un mensaje.

Lógica a implementar:

- Verificar en customer si el cliente está activo (active = 1). Si no está activo: p_codigo_resultado = 101 y p_mensaje = 'Cliente Inactivo'.
- Si el cliente está activo, contar cuántas copias de la película existen en la tabla inventory para esa tienda (store_id).
- Si hay 0 copias: p_codigo_resultado = 102 y p_mensaje = 'Sin Stock en esta Sucursal'.
- Si pasa ambas validaciones: p_codigo_resultado = 200 y p_mensaje = 'Alquiler Habilitado'.

```
CALL sp_validar_alquiler_express(1, 10, 1, @codigo, @msg);
SELECT @codigo, @msg;
```

EJERCICIO MESA 3

Calculador de Tarifa por Clasificación (Rating)

Crear el procedimiento *sp_calcular_precio_rating*. Debe recibir el ID de una película y devolver el precio final ajustado y un mensaje explicativo

Lógica a implementar:

- Consultar el rental_rate y el rating de la película en la tabla film.
- Evaluar el rating usando la estructura CASE:
  - 'G': Aplicar un 10%de descuento al rental_rate (Leyenda: 'Promoción Infantil').
  - 'PG' o 'PG-13': Mantener el rental_rate original (Leyenda: 'Tarifa Estándar').
  - 'NC-17' o 'R': Aplicar un 15%de recargo al rental_rate (Leyenda: 'Tarifa Adultos').

```
CALL sp_calcular_precio_rating(5, @precio, @leyenda);
SELECT @precio, @leyenda;
```

EJERCICIO MESA 4

Generador de Cupones de Descuento

Crear el procedimiento *sp_generar_cupones_cliente*. Debe recibir el ID de un cliente y la cantidad de cupones a generar

- **Lógica a implementar:**
- Declarar una variable contador v_i INT DEFAULT 1.
- Utilizar un bucle WHILE que itere desde 1 hasta p_cantidad.
- En cada iteración, mostrar/imprimir (o insertar en una tabla de prueba si la crean) un código de cupón con el formato: 'CUPON-CLIENTE-[customer_id]-NRO-[v_i]'.
- Incrementar el contador en cada paso.

```
CALL sp_generar_cupones_cliente(10, 5);
```

Modificación de Stored Procedures

- La instrucción ALTER PROCEDURE **existe**, pero **NO sirve para cambiar el cuerpo (BEGIN...END) ni los parámetros** del procedimiento. Solo sirve para cambiar metadatos (como comentarios o permisos).
- Para modificar la lógica de un Stored Procedure en SQL, la forma estándar es **eliminarlo y volverlo a crear**:

```
-- 1. Eliminar la versión anterior si existe
DROP PROCEDURE IF EXISTS nombre_procedimiento;

-- 2. Volverlo a crear con la nueva lógica
DELIMITER //

CREATE PROCEDURE nombre_procedimiento(...)
BEGIN
   -- Nueva lógica modificada
END //

DELIMITER ;
```

EJERCICIO MODIFICACION

Grupo 1: Modificar sp_evaluar_fidelidad_cliente

Nuevos Requerimientos:

- Agregar una nueva categoría **'PLATINUM'** para clientes con **más de 40 alquileres**, la cual otorgará un 0.30**(**30%**) de descuento**.
- **Validación de cliente inactivo:** Antes de calcular el descuento, verificar en la tabla customer si el cliente está activo (active = 1). Si está inactivo (active = 0), el procedimiento debe devolver la categoría 'INACTIVO' y 0.00de descuento, sin importar cuántas películas haya alquilado antes.

Grupo 2: Modificar sp_calcular_precio_rating

Nuevos Requerimientos:

- Agregar un **nuevo parámetro de entrada** IN p_es_fin_de_semana BOOLEAN.
- Modificar la estructura CASE para que, si p_es_fin_de_semana es TRUE (1), se aplique un **recargo extra del** 10%sobre el precio final calculado para cualquier clasificación (rating).
- Ajustar la leyenda de salida para que indique si se aplicó la **"Tarifa Fin de Semana"** o la **"Tarifa Día de Semana"**.

EJERCICIO MODIFICACION

Grupo 3: Modificar sp_generar_cupones_cliente

Nuevos Requerimientos:

- **Límite de seguridad:** Si el parámetro p_cantidad es mayor a 10, el procedimiento no debe ejecutar el bucle y debe mostrar un mensaje de error: 'Error: No se pueden generar más de 10 cupones a la vez'.
- **Formato dinámico del cupón:** Dentro del bucle WHILE, si la cuota o cupón es par (ej. cupón 2, 4, 6...), asignarle el texto 'CUPON-PAR-[customer_id]-NRO-[v_i]'; si es impar, mantener el formato original 'CUPON-CLIENTE-[customer_id]-NRO-[v_i]'.

Grupo 4: Modificar sp_validar_alquiler_express

Nuevos Requerimientos:

- **Validación de deudores (Nueva regla):** Antes de verificar el stock, el procedimiento debe consultar si el cliente tiene alguna película pendiente de devolución (es decir, registros en la tabla rental donde customer_id = p_customer_id y return_date IS NULL).
- Si el cliente tiene **más de 2 películas sin devolver**, el procedimiento debe frenar la validación y devolver p_codigo_resultado = 103 y p_mensaje = 'Cliente con exceso de películas no devueltas'.
- Si debe 2 o menos, continúa con la verificación normal de stock.